BÀI TẬP CSP VÀ GAME TRONG AI

Họ và tên	Đặng Ngọc Như Ý
MSSV / Lớp	3123410438 / DNT1241

Mục tiêu học tập

• Mô hình hóa và giải các bài toán thỏa mãn ràng buộc (CSP).

• Vận dụng Forward Checking, AC-3, MRV và LCV trong CSP.

• Phân tích Minimax, Alpha-Beta Pruning và Expectimax trong trò chơi đối kháng.

• Nhận biết ảnh hưởng của thứ tự duyệt và cắt tỉa đến hiệu quả tìm kiếm.

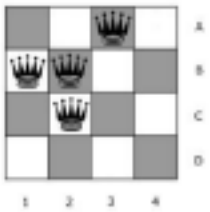
• Vận dụng các kỹ thuật trên vào bài toán 4-Queens, điều phối đường băng và Magic Square.

Nội dung	Kỹ thuật	Ý nghĩa chính
CSP	Forward Checking	Loại giá trị xung đột sau mỗi phép gán
CSP	AC-3	Duy trì tính nhất quán cung
CSP	MRV / LCV	Chọn biến khó trước / chọn giá trị ít ràng buộc nhất
Game	Minimax / Alpha-Beta	Tìm quyết định tối ưu và cắt tỉa nhánh không ảnh hưởng
Game	Expectimax	Mô hình hóa nút ngẫu nhiên bằng kỳ vọng

Bài 1. Bài toán 4-Queens và Hill Climbing

Hình dưới trình bày giả mã thuật toán hill-climbing cho bài toán 4 quân hậu và một trạng thái khởi đầu. Các quân hậu di chuyển như trong cờ vua: có thể đi nhiều ô theo hàng, cột hoặc đường chéo miễn là đường đi không bị chặn.

```
function 4-QUEENS-HILL-CLIMBING(problem)
    current_state = problem.INITIAL-STATE
    loop do
        neighbor = state with lowest number of conflicts between queens obtained by
                   making one legal chess move from current_state
        if neighbor.num_conflicts >= current_state.num_conflicts
            return current_state
        current_state = neighbor
```



Hình 1. Mã giả Hill Climbing và trạng thái khởi đầu của bài toán 4-Queens

a) Hãy ghi trạng thái của từng quân hậu sau một vòng lặp của thuật toán. Một vòng lặp tương đương với việc di chuyển một quân hậu.

Sau một vòng lặp, chuyển quân hậu từ B2 sang D4. Trạng thái mới: A3, B1, C2, D4; còn 1 cặp xung đột là B1–C2 (cùng đường chéo).....

b) Thuật toán hill-climbing ở trên có dừng với trạng thái khởi đầu đã cho hay không? Nếu có, trạng thái cuối thuộc loại nào? Chọn tất cả phương án phù hợp.

☐ A) Không dừng

☑ B) Cực trị cục bộ (local optimum)

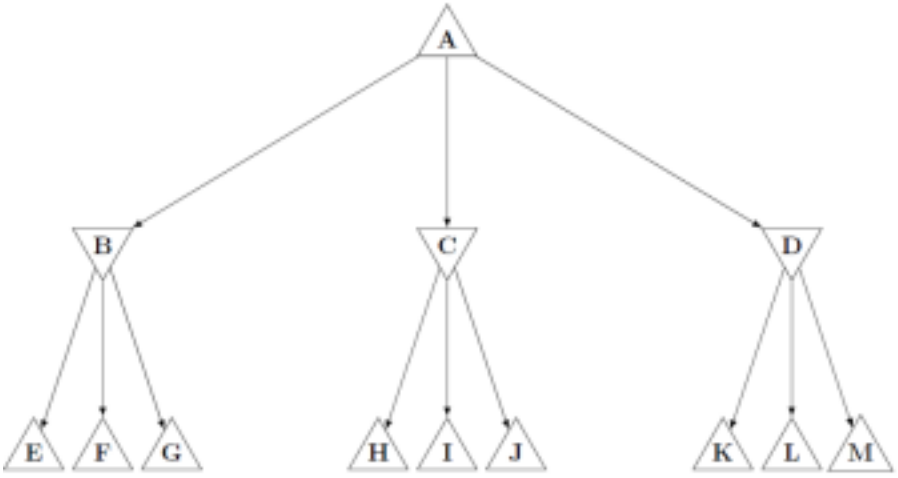
☐ C) Cực trị toàn cục (global optimum)

☐ D) Vai bằng (shoulder)

☐ E) Không thuộc các trường hợp trên

Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening

Xét cây trò chơi trong Hình 2. Gốc A là nút MAX; B, C, D là các nút MIN. Bảng bên phải cho giá trị hàm đánh giá f(s) của các trạng thái. Iterative deepening có thể kết hợp với cắt tỉa để tăng khả năng pruning và giảm thời gian tìm kiếm.



Evaluation Function	
State, s	$f(s)$
B	4
C	14
D	19
E	5
F	7
G	8
H	17
I	16
J	15
K	23
L	20
M	26

Hình 2. Cây trò chơi và bảng giá trị hàm đánh giá

Thực hiện theo hai bước: (1) sắp thứ tự các cây con B, C, D dựa trên f(B), f(C), f(D), từ tốt nhất đến xấu nhất theo góc nhìn của nút cha; (2) áp dụng Alpha-Beta Pruning theo thứ tự mới, giới hạn độ sâu đến các nút cháu E, F, G, ..., và dùng giá trị đánh giá tại các nút cháu.

a) Thứ tự các cây con sau khi sắp xếp là:

☐ B → C → D

☐ C → D → B

☑ D → C → B

b) Với thứ tự mới, hãy liệt kê các nút KHÔNG được thăm do bị cắt tỉa. Các nút cháu trong mỗi cây con vẫn được xét từ trái sang phải.

Duyệt D trước: D = min(23, 20, 26) = 20 nên α tại A bằng 20. Ở C, H = 17 làm β = 17 ≤ α, cắt I,J. Ở B, E = 5 làm β = 5 ≤ α, cắt F, G. Các nút không được thăm: I, J, F, G.....

c) Alpha-Beta Pruning chuẩn đảm bảo cho cùng nước đi với Minimax đầy đủ. Phương pháp iterative deepening kết hợp pruning như mô tả ở trên có luôn đảm bảo trả về cùng nước đi với Minimax trên toàn bộ cây hay không? Giải thích ngắn gọn.

☐ Có

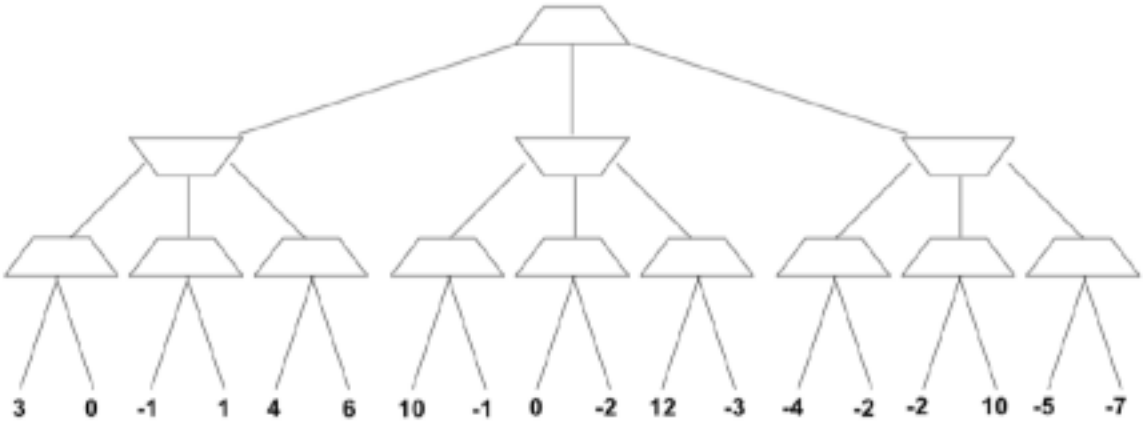
☑ Không

Alpha-Beta duyệt đủ độ sâu bảo toàn kết quả Minimax, nhưng iterative deepening nếu dừng ở giới hạn độ sâu chưa đủ có thể chọn nước đi khác do các nút biên được đánh giá bằng heuristic……...

Bài 3. Adversarial Search - Alpha-Beta Pruning

Xét cây trò chơi dưới đây, trong đó nút gốc là MAX. Áp dụng Alpha-Beta Pruning và duyệt các nút kế tiếp từ trái

sang phải.



Hình 3. Cây trò chơi dùng cho Alpha-Beta Pruning

Yêu cầu:

• Ghi giá trị α và β tại mỗi nút trong quá trình duyệt.

• Ghi giá trị được trả về tại mỗi nút trung gian.

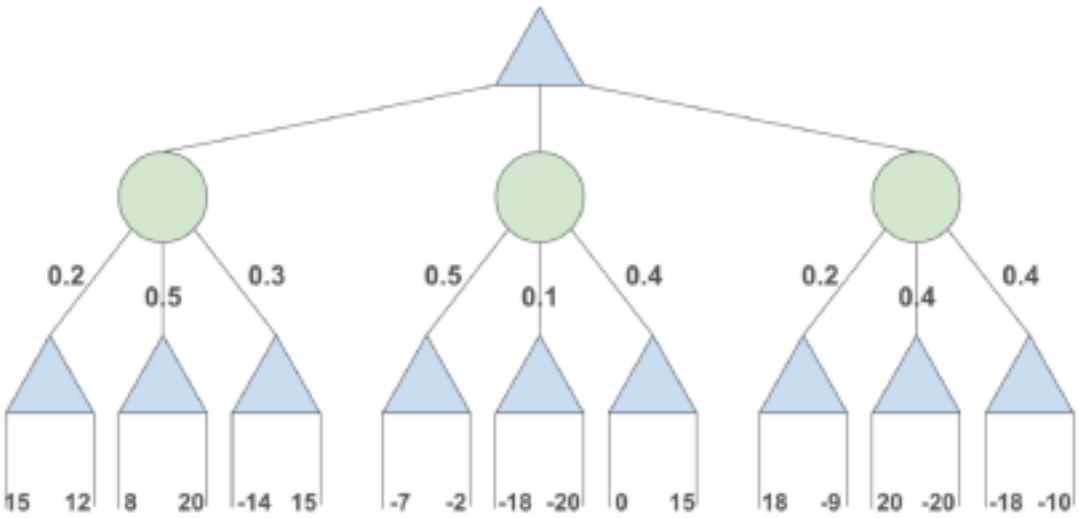
• Đánh dấu X lên các cạnh hoặc nhánh bị cắt tỉa.

• Cho biết giá trị Minimax tại nút gốc và nước đi tối ưu của MAX.

Kết quả Alpha-Beta (duyệt trái sang phải): Gốc MAX trả về 1, chọn nhánh trái. Nút MIN trái trả về 1; MIN giữa trả về 0; MIN phải trả về −5. Cắt lá 6 ở nhánh trái và cắt toàn bộ cây con có lá 12, −3 ở nhánh giữa. Diễn tiến cận: nhánh trái, MAX đầu trả 3, MAX thứ hai trả 1; tại MAX thứ ba, xét 4 thì $\alpha=4\geq\beta=1$ nên cắt 6. Nhánh giữa: MAX đầu trả 10; MAX thứ hai trả 0, khiến $\beta = 0 \leq \alpha$ gốc = 1, cắt nhánh có 12, −3. Nhánh phải trả −5……

Bài 4. Expectimax và Chance Nodes

Xét cây trò chơi trong Hình 4. Nút gốc là MAX (tam giác màu xanh), các nút tròn màu xanh lá là chance node. Xác suất của mỗi nhánh được ghi trên cạnh. Các nút MAX ở tầng dưới chọn giá trị lớn nhất trong các lá của chúng.



Hình 4. Cây Expectimax với chance nodes

a) Duyệt các nút con từ trái sang phải. Hãy tính giá trị được trả về tại từng nút MAX, từng chance node và tại nút gốc.

Các nút MAX tầng dưới lần lượt trả về:

max(15, 12) = 15; max(8, 20) = 20; max(−14, 15) = 15; max(−7, −2) = −2; max(−18, −20) = −18; max(0, 15) = 15; max(18, −9) = 18; max(20, −20) = 20; max(−18, −10) = −10……..

Các chance node:.....

Trái = 0,2 × 15 + 0,5 × 20 + 0,3 × 15 = 17,5;.....

Giữa = 0,5 × (−2) + 0,1 × (−18) + 0,4 × 15 = 3,2;.....

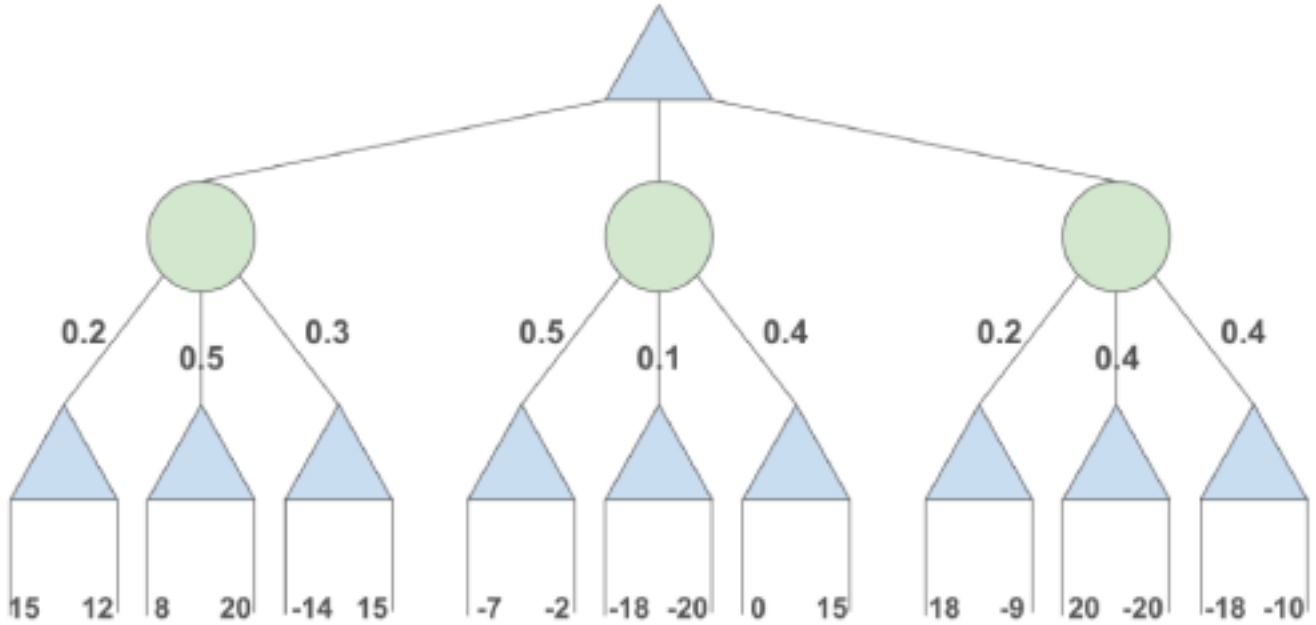
Phải = 0,2 × 18 + 0,4 × 20 + 0,4 × (−10) = 7,6…….

Gốc MAX chọn 17,5 (nhánh trái).....

b) Nếu chưa biết thêm giới hạn nào về giá trị lá, những nút nào trong cây trên có thể được cắt tỉa mà vẫn đảm bảo kết quả chính xác? Giải thích.

Không có cận trên/dưới cho các lá chưa xét nên không thể đảm bảo cắt tỉa một nhánh chance nào mà vẫn giữ chính xác kỳ vọng. Giá trị các lá chưa xét có thể làm thay đổi kỳ vọng và lựa chọn ở gốc; do đó cần xét đầy đủ các nút để bảo đảm kết quả chính xác. c) Giả sử mỗi nút lá đều có giá trị nằm trong đoạn [-20, 20]. Hãy giải lại cây và cắt tỉa các nhánh không cần thiết; vẫn duyệt từ trái sang phải……

c) Giả sử mỗi nút lá đều có giá trị nằm trong đoạn [-20, 20]. Hãy giải lại cây và cắt tỉa các nhánh không cần thiết; vẫn duyệt từ trái sang phải.



Hình 5. Cây Expectimax dùng cho câu 4(c)

Nhánh trái có giá trị 17,5. Ở nhánh giữa, nút MAX đầu = −2 với xác suất 0,5;.....

Cận trên kỳ vọng = 0,5 × (−2) + (0,1 + 0,4) × 20 = 9 < 17,5, nên cắt hai nút MAX còn lại của nhánh giữa…….

Ở nhánh phải, sau nút MAX đầu = 18, cận trên = 19,6; sau nút MAX thứ hai =20, cận trên vẫn 19,6 nên chưa thể cắt nút cuối……...

Xét nút cuối = −10, nhánh phải có kỳ vọng 7,6. Gốc vẫn chọn nhánh trái với 17,5……...

Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking khác AC-3 ở điểm nào? So sánh phạm vi lan truyền ràng buộc, thời điểm thực hiện và chi phí tính toán.

Forward Checking chỉ loại các giá trị không tương thích khỏi miền của các biến chưa gán kề với biến vừa gán; nó được thực hiện sau mỗi phép gán và thường có chi phí mỗi bước thấp hơn. AC-3 duy trì tính nhất quán cung trên các cung của đồ thị; khi miền bị thu hẹp, các cung liên quan được đưa lại vào hàng đợi để lan truyền tiếp qua nhiều biến. AC-3 thường phát hiện mâu thuẫn sâu hơn nhưng tốn thêm chi phí kiểm tra cung……..

b) Vì sao các heuristic sau thường được dùng trong bài toán CSP?

i) Minimum Remaining Values (MRV)

MRV chọn biến chưa gán có ít giá trị hợp lệ nhất. Biến có ít lựa chọn thường dễ dẫn đến bế tắc; gán biến đó trước giúp phát hiện mâu thuẫn sớm và giảm việc tìm kiếm các nhánh vô ích…….

.....

ii) Least Constraining Value (LCV)

LCV chọn giá trị loại bỏ ít lựa chọn nhất khỏi miền của các biến chưa gán có liên quan. Cách này giữ lại nhiều khả năng cho các biến còn lại, thường giúp tìm nghiệm dễ hơn và giảm nhu cầu quay lui

.....

Bài 6. CSP - Điều phối không lưu

Có năm máy bay A, B, C, D và E cùng hai đường băng: quốc tế và nội địa. Cần xếp cho mỗi máy bay một khung giờ và một đường băng để hạ cánh hoặc cất cánh. Mỗi đường băng có bốn khung giờ 1, 2, 3, 4. Bài toán phải thỏa các ràng buộc sau:

• Máy bay B bị hỏng một động cơ và phải hạ cánh ở khung giờ 1.

• Máy bay D chỉ có thể đến sân bay để hạ cánh từ khung giờ 3 trở đi.

• Máy bay A sắp hết nhiên liệu và chỉ có thể chờ tối đa đến khung giờ 2.

• Máy bay D phải hạ cánh trước khi máy bay C cất cánh vì có hành khách nối chuyến từ D sang C. • Không có hai máy bay nào được đặt cùng một khung giờ trên cùng một đường băng. a) Hãy mô hình hóa bài toán dưới dạng CSP: xác định biến, miền giá trị và các ràng buộc đơn/nguyên (unary) và nhị phân (binary). Viết ràng buộc bằng ký hiệu toán học hoặc logic.

Biến: $X_i = (T_i, R_i)$, với $i \in \{A, B, C, D, E\}$; $T_i \in \{1, 2, 3, 4\}$, $R_i \in$ {Quốc tế, Nội địa}.....

Ràng buộc unary: $T_B = 1$; $T_D \in \{3,4\}$; $T_A \in \{1,2\}$. Ràng buộc nhị phân: $T_D < T_C$; với mọi $i \neq j$, nếu $R_i = R\square$ thì $T_i \neq T\square$…….

Bổ sung hai ràng buộc sau:

• A, B, C là các chuyến bay quốc tế và chỉ dùng đường băng quốc tế.

• D, E là các chuyến bay nội địa và chỉ dùng đường băng nội địa.

b) Với hai ràng buộc bổ sung, loại đường băng không cần nằm trong miền giá trị vì đã được xác định bởi biến. Hãy xác định miền mới và vẽ đồ thị ràng buộc của bài toán.

Do đường băng đã cố định: $A \in \{1, 2\}$; $B = \{1\}$; $C \in \{1, 2, 3, 4\}$; $D \in \{3, 4\}$; $E \in \{1, 2, 3, 4\}$. Đồ thị ràng

buộc có cạnh A–B, A–C, B–C (các chuyến quốc tế khác giờ), D–E (các chuyến nội địa khác giờ), và D–C (T_D < T_C).....

c) Sau khi áp dụng arc consistency, miền của các biến còn lại là gì? Bắt đầu bằng việc áp dụng các ràng buộc unary. Hãy gạch bỏ những giá trị bị loại khỏi miền.

Áp dụng unary: A = {1,2}, B = {1}, C = {1,2,3,4}, D = {3,4}, E = {1,2,3,4}.....

Từ B = 1 suy ra A,C ≠ 1; do A chỉ còn 2 nên C ≠ 2…….

Từ D < C, D = 4 bị loại vì không có C lớn hơn 4; C=3 bị loại vì không có D nhỏ hơn 3……

Do đó A = {2}, B = {1}, C = {4}, D = {3}.....

Vì E cùng đường băng với D nên E ≠ 3, suy ra E = {1,2,4}.....

d) Thay vì duy trì arc consistency toàn cục, hãy dùng Forward Checking kết hợp Backtracking Search. Chọn biến theo MRV; khi hòa, chọn ký tự nhỏ hơn trước; chọn giá trị nhỏ hơn trước. Hãy liệt kê các cặp (biến, giá trị) theo đúng thứ tự phát sinh, kể cả các phép gán bị hoàn tác khi gặp dead end. Áp dụng ràng buộc unary trước khi bắt đầu.

A	1	2	3	4
B	1	2	3	4
C	1	2	3	4
D	1	2	3	4
E	1	2	3	4

Áp dụng unary trước: A = {1,2}, B = {1}, C = {1,2,3,4}, D = {3,4}, E = {1,2,3,4}.....

MRV và phá hòa theo chữ cái: (B,1) → (A,2) → (C,3).....

Khi C = 3, D phải nhỏ hơn 3 nhưng D ∈ {3, 4}, miền D rỗng nên dead end và hoàn tác (C, 3).....

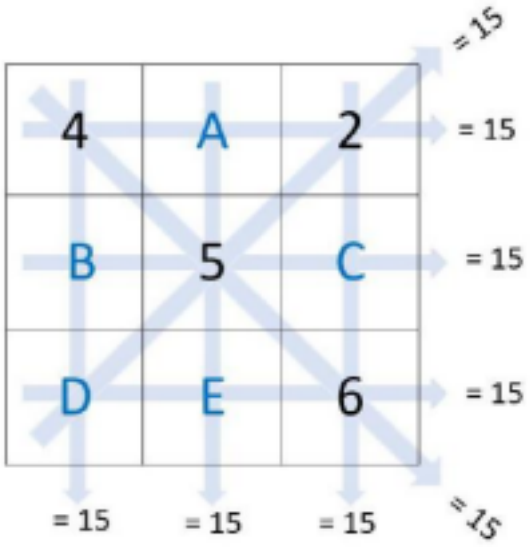
Thử tiếp (C, 4), rồi (D, 3) do D < C, cuối cùng (E, 1) theo giá trị nhỏ nhất còn hợp lệ…….

Thứ tự gán: (B, 1), (A, 2), (C, 3) [hoàn tác], (C, 4), (D, 3), (E, 1).....

Nghiệm: A = 2, B = 1, C = 4, D = 3, E = 1……

Bài 7. CSP - Magic Square

Magic Square là bảng n × n trong đó mỗi ô chứa một giá trị khác nhau thuộc {1, ..., n^2}; tổng của mỗi hàng, mỗi cột và hai đường chéo đều bằng nhau. Với bảng 3 × 3 dưới đây, tổng cần đạt là 15. Một số ô đã được cho trước, các ô ký hiệu chữ cái cần được xác định.



Hình 6. Bài toán Magic Square 3 × 3

a) Hãy mô hình hóa bài toán dưới dạng CSP: biến, miền giá trị và các ràng buộc. Có thể bỏ qua việc tạo biến cho các ô đã biết.

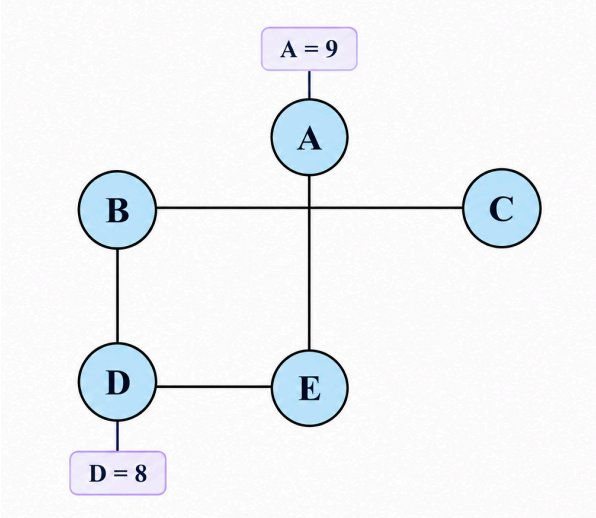
Biến chưa biết: A, B, C, D, E. Miền ban đầu là các số chưa dùng trong {1,…,9}, loại các số cố định 4, 5, 2, 6; các ô phải khác nhau (alldiff).....

Ràng buộc tổng: 4 + A + 2 = 15; B + 5 + C = 15; D + E + 6 = 15; 4 + B + D = 15; A + 5 + E = 15;.....

Các cột còn lại và hai đường chéo đều có tổng 15…….

Từ đường chéo D + 5 + 2 = 15 => D = 8 => A = 9……..

b) Vẽ đồ thị ràng buộc nhị phân. Để đơn giản, có thể bỏ qua ràng buộc alldiff khi vẽ đồ thị



c) Sử dụng đồ thị ràng buộc nhị phân và thuật toán AC-3 để thu hẹp miền, sau đó tìm một nghiệm cho Magic Square.

AC-3 và các ràng buộc tổng cho nghiệm: A = 9, D = 8; A + 5 + E = 15……

⇒ E = 1; 4 + B + D = 15……...

⇒ B = 3; B + 5 + C = 15……...

⇒ C = 7……...

Magic Square hoàn chỉnh:……...

4	**9**	**2**
3	**5**	**7**
8	**1**	**6**

Bài 8. MRV và LCV trong thực hành

Ba sinh viên AJ, Sunny và Ellyse cần chọn mỗi người đúng một màu sơn sao cho không có hai người chọn cùng màu. Các tập màu có sẵn:

• AJ = {Đỏ, Vàng, Xanh lá}

• Sunny = {Đỏ, Vàng, Hồng}

• Ellyse = {Đỏ, Hồng}

a) Dùng MRV để chọn biến và LCV để chọn giá trị. Nếu hòa, phá hòa theo thứ tự cầu vồng. Hỏi thuật toán phải backtrack về phép gán trước đó bao nhiêu lần? Hãy ghi lại thứ tự gán.

MRV chọn Ellyse vì miền {Đỏ, Hồng} nhỏ nhất. LCV chọn Hồng vì chỉ loại Hồng khỏi miền Sunny; chọn Đỏ sẽ loại Đỏ khỏi cả AJ và Sunny. Gán Ellyse = Hồng. Sunny còn {Đỏ, Vàng}; phá hòa theo thứ tự cầu vồng chọn Đỏ trước, rồi AJ = Vàng……...

Thứ tự gán: Ellyse = Hồng → Sunny = Đỏ → AJ = Vàng……

Số lần backtrack: 0…….

b) Thay MRV bằng Maximum Remaining Values và thay LCV bằng Most Constraining Value. Hỏi số lần backtrack là bao nhiêu? So sánh với câu (a) và giải thích vì sao.

AJ: {Đỏ, Vàng, Xanh lá} → 3 giá trị…….

Sunny: {Đỏ, Vàng, Hồng} → 3 giá trị……...

Ellyse: {Đỏ, Hồng} → 2 giá trị……..

AJ = Đỏ: ràng buộc Sunny và Ellyse……..

AJ = Vàng: ràng buộc Sunny……..

AJ = Xanh lá: không ảnh hưởng đến hai biến còn lại……

=> Chọn AJ = Đỏ => Sunny: {Vàng, Hồng} → 2 giá trị, Ellyse: {Hồng} → 1 giá trị…….

Sunny = Hồng → Ellyse không còn giá trị hợp lệ → bế tắc, quay lui……...

Sunny = Vàng → Ellyse vẫn có thể chọn Hồng…….

=> Chọn Sunny = Vàng => Ellyse = Hồng……

Vậy AJ = Đỏ, Sunny = Vàng, Ellyse = Hồng…….

Thứ tự gán: AJ = Đỏ → Sunny = Hồng (bế tắc, quay lui) → Sunny = Vàng → Ellyse = Hồng……

Số lần quay lui: 1 lần……...

--- Hết ---